Weekly Assignment 3: Security & JWT Implementation (Real-Time Chat Application)

This document contains the source code for the **Security Configurations**, **JWT Utilities**, **User Details Service**, and **Authentication API Controllers** for the Real-Time Chat Application backend, along with a **JWT Lifecycle Flow Diagram** and a **Verification & Testing Plan** detailing system security endpoints.

---

1. Maven Dependencies

The following dependency block is configured in the Maven build system to pull in Spring Security, validation, and JWT APIs and implementations:

```
<!-- Spring Security Starter -->
<dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-security</artifactId>
</dependency>

<!-- JSON Web Token (JWT) API -->
<dependency>
   <groupId>io.jsonwebtoken</groupId>
   <artifactId>jjwt-api</artifactId>
   <version>0.12.5</version>
</dependency>

<!-- JWT Implementation Runtime -->
<dependency>
   <groupId>io.jsonwebtoken</groupId>
   <artifactId>jjwt-impl</artifactId>
   <version>0.12.5</version>
   <scope>runtime</scope>
</dependency>

<!-- JJWT Jackson integration for JSON serialization/deserialization -->
<dependency>
   <groupId>io.jsonwebtoken</groupId>
   <artifactId>jjwt-jackson</artifactId>
   <version>0.12.5</version>
   <scope>runtime</scope>
</dependency>
```

---

2. Spring Security Config

WebSecurityConfig.java

Configures bean definitions, Stateless Session Policy, password encoder, exception handlers, and endpoints permitting/requiring authentication.

```
package com.chatapp.security;

import com.chatapp.security.jwt.AuthEntryPointJwt;
import com.chatapp.security.jwt.AuthTokenFilter;
import com.chatapp.security.services.UserDetailsServiceImpl;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.dao.DaoAuthenticationProvider;
import org.springframework.security.config.annotation.authentication.configuration.AuthenticationConf
import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
import org.springframework.web.cors.CorsConfiguration;
import org.springframework.web.cors.CorsConfigurationSource;
import org.springframework.web.cors.UrlBasedCorsConfigurationSource;
import java.util.List;

@Configuration
@EnableWebSecurity
@EnableMethodSecurity
public class WebSecurityConfig {

    @Autowired
    UserDetailsServiceImpl userDetailsService;

    @Autowired
    private AuthEntryPointJwt unauthorizedHandler;

    @Bean
    public AuthTokenFilter authenticationJwtTokenFilter() {
        return new AuthTokenFilter();
    }

    @Bean
    public DaoAuthenticationProvider authenticationProvider() {
```

```
        DaoAuthenticationProvider authProvider = new DaoAuthenticationProvider();
        authProvider.setUserDetailsService(userDetailsService);
        authProvider.setPasswordEncoder(passwordEncoder());
        return authProvider;
    }

    @Bean
    public AuthenticationManager authenticationManager(AuthenticationConfiguration authConfig) throws
        return authConfig.getAuthenticationManager();
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    public CorsConfigurationSource corsConfigurationSource() {
        CorsConfiguration configuration = new CorsConfiguration();
        configuration.setAllowedOriginPatterns(List.of("*"));
        configuration.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "OPTIONS"));
        configuration.setAllowedHeaders(List.of("Authorization", "Content-Type", "Cache-Control"));
        configuration.setAllowCredentials(true);
        UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
        source.registerCorsConfiguration("/**", configuration);
        return source;
    }

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http.csrf(csrf -> csrf.disable())
            .cors(cors -> cors.configurationSource(corsConfigurationSource()))
            .exceptionHandling(exception -> exception.authenticationEntryPoint(unauthorizedHandler))
            .sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELES
            .authorizeHttpRequests(auth ->
                auth.requestMatchers("/api/auth/**").permitAll()
                    .requestMatchers("/h2-console/**").permitAll()
                    .requestMatchers("/ws/**").permitAll()
                    .anyRequest().authenticated()
            );

        // Required for accessing H2 database console within iframes
        http.headers(headers -> headers.frameOptions(frame -> frame.sameOrigin()));

        http.authenticationProvider(authenticationProvider());
        http.addFilterBefore(authenticationJwtTokenFilter(), UsernamePasswordAuthenticationFilter.clas

        return http.build();
    }
}
```

---

3. JWT Utility & Filters

JwtUtils.java

Contains utilities for generating new JSON Web Tokens, parsing user claims from active tokens, and verifying signature authenticity.

```
package com.chatapp.security.jwt;

import com.chatapp.security.services.UserDetailsImpl;
import io.jsonwebtoken.*;
import io.jsonwebtoken.security.Keys;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.security.core.Authentication;
import org.springframework.stereotype.Component;
import javax.crypto.SecretKey;
import java.nio.charset.StandardCharsets;
import java.util.Date;

@Component
public class JwtUtils {
   private static final Logger logger = LoggerFactory.getLogger(JwtUtils.class);

   @Value("${chatapp.jwtSecret}")
   private String jwtSecret;

   @Value("${chatapp.jwtExpirationMs}")
   private int jwtExpirationMs;

   private SecretKey getSignKey() {
       byte[] keyBytes = this.jwtSecret.getBytes(StandardCharsets.UTF_8);
       return Keys.hmacShaKeyFor(keyBytes);
   }

   public String generateJwtToken(Authentication authentication) {
       UserDetailsImpl userPrincipal = (UserDetailsImpl) authentication.getPrincipal();
       return Jwts.builder()
               .subject(userPrincipal.getUsername())
               .issuedAt(new Date())
               .expiration(new Date((new Date()).getTime() + jwtExpirationMs))
               .signWith(getSignKey())
               .compact();
   }

   public String getUserNameFromJwtToken(String token) {
       return Jwts.parser()
               .verifyWith(getSignKey())
```

```
                .build()
                .parseSignedClaims(token)
                .getPayload()
                .getSubject();
    }

    public boolean validateJwtToken(String authToken) {
        try {
            Jwts.parser()
                .verifyWith(getSignKey())
                .build()
                .parseSignedClaims(authToken);
            return true;
        } catch (MalformedJwtException e) {
            logger.error("Invalid JWT token: {}", e.getMessage());
        } catch (ExpiredJwtException e) {
            logger.error("JWT token is expired: {}", e.getMessage());
        } catch (UnsupportedJwtException e) {
            logger.error("JWT token is unsupported: {}", e.getMessage());
        } catch (IllegalArgumentException e) {
            logger.error("JWT claims string is empty: {}", e.getMessage());
        }
        return false;
    }
}
```

AuthTokenFilter.java

Custom servlet interceptor filter extracting JWT authorization headers and attaching the security authentication instance to the context of the HTTP thread.

```
package com.chatapp.security.jwt;

import org.springframework.lang.NonNull;
import com.chatapp.security.services.UserDetailsServiceImpl;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.web.authentication.WebAuthenticationDetailsSource;
import org.springframework.util.StringUtils;
import org.springframework.web.filter.OncePerRequestFilter;
import java.io.IOException;
```

```
public class AuthTokenFilter extends OncePerRequestFilter {
   private static final Logger logger = LoggerFactory.getLogger(AuthTokenFilter.class);

   @Autowired
   private JwtUtils jwtUtils;

   @Autowired
   private UserDetailsServiceImpl userDetailsService;

   @Override
   protected void doFilterInternal(@NonNull HttpServletRequest request, @NonNull HttpServletResponse
           throws ServletException, IOException {
       try {
           String jwt = parseJwt(request);
           if (jwt != null && jwtUtils.validateJwtToken(jwt)) {
               String username = jwtUtils.getUserNameFromJwtToken(jwt);

               UserDetails userDetails = userDetailsService.loadUserByUsername(username);
               UsernamePasswordAuthenticationToken authentication =
                       new UsernamePasswordAuthenticationToken(userDetails, null, userDetails.getAuth
               authentication.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));

               SecurityContextHolder.getContext().setAuthentication(authentication);
           }
       } catch (Exception e) {
           logger.error("Cannot set user authentication: {}", e);
       }

       filterChain.doFilter(request, response);
   }

   private String parseJwt(HttpServletRequest request) {
       String headerAuth = request.getHeader("Authorization");

       if (StringUtils.hasText(headerAuth) && headerAuth.startsWith("Bearer ")) {
           return headerAuth.substring(7);
       }

       return null;
   }
}
```

---

4. User Details Service

UserDetailsServiceImpl.java

Overrides the default `UserDetailsService` to link the Spring database `UserRepository` lookup queries to security operations.

```
package com.chatapp.security.services;

import com.chatapp.model.User;
import com.chatapp.repository.UserRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
public class UserDetailsServiceImpl implements UserDetailsService {

   @Autowired
   UserRepository userRepository;

   @Override
   @Transactional
   public UserDetails loadUserByUsername(String username) throws UsernameNotFoundException {
       User user = userRepository.findByUsername(username)
               .orElseThrow(() -> new UsernameNotFoundException("User Not Found with username: " + us

       return UserDetailsImpl.build(user);
   }
}
```

UserDetailsImpl.java

Class implementation mapping the Hibernate entity fields into variables expected by Security context interfaces.

```
package com.chatapp.security.services;

import com.chatapp.model.User;
import com.fasterxml.jackson.annotation.JsonIgnore;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.core.userdetails.UserDetails;
import java.util.Collection;
import java.util.Collections;
import java.util.Objects;

public class UserDetailsImpl implements UserDetails {
   private static final long serialVersionUID = 1L;

   private Long id;
   private String username;
```

```
    private String email;
    private String avatarUrl;

    @JsonIgnore
    private String password;

    public UserDetailsImpl(Long id, String username, String email, String password, String avatarUrl)
        this.id = id;
        this.username = username;
        this.email = email;
        this.password = password;
        this.avatarUrl = avatarUrl;
    }

    public static UserDetailsImpl build(User user) {
        return new UserDetailsImpl(
                user.getId(),
                user.getUsername(),
                user.getEmail(),
                user.getPassword(),
                user.getAvatarUrl()
        );
    }

    public Long getId() {
        return id;
    }

    public String getEmail() {
        return email;
    }

    public String getAvatarUrl() {
        return avatarUrl;
    }

    @Override
    public Collection<? extends GrantedAuthority> getAuthorities() {
        return Collections.emptyList();
    }

    @Override
    public String getPassword() {
        return password;
    }

    @Override
    public String getUsername() {
        return username;
    }
```

```
    @Override
    public boolean isAccountNonExpired() {
        return true;
    }

    @Override
    public boolean isAccountNonLocked() {
        return true;
    }

    @Override
    public boolean isCredentialsNonExpired() {
        return true;
    }

    @Override
    public boolean isEnabled() {
        return true;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        UserDetailsImpl user = (UserDetailsImpl) o;
        return Objects.equals(id, user.id);
    }
}
```

---

5. DTOs & Controller

AuthController.java

Exposes REST endpoints to authenticate usernames, hash new credentials, log out users, and update active chat presence properties.

```
package com.chatapp.controller;

import com.chatapp.dto.JwtResponse;
import com.chatapp.dto.LoginRequest;
import com.chatapp.dto.SignupRequest;
import com.chatapp.model.User;
import com.chatapp.model.UserStatus;
import com.chatapp.repository.UserRepository;
import com.chatapp.security.jwt.JwtUtils;
import com.chatapp.security.services.UserDetailsImpl;
```

```
import jakarta.validation.Valid;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.web.bind.annotation.*;
import java.util.HashMap;
import java.util.Map;

@RestController
@RequestMapping("/api/auth")
public class AuthController {

   @Autowired
   AuthenticationManager authenticationManager;

   @Autowired
   UserRepository userRepository;

   @Autowired
   PasswordEncoder encoder;

   @Autowired
   JwtUtils jwtUtils;

   @PostMapping("/login")
   public ResponseEntity<?> authenticateUser(@Valid @RequestBody LoginRequest loginRequest) {
       Authentication authentication = authenticationManager.authenticate(
               new UsernamePasswordAuthenticationToken(loginRequest.getUsername(), loginRequest.getPa

       SecurityContextHolder.getContext().setAuthentication(authentication);
       String jwt = jwtUtils.generateJwtToken(authentication);

       UserDetailsImpl userDetails = (UserDetailsImpl) authentication.getPrincipal();

       userRepository.findByUsername(userDetails.getUsername()).ifPresent(user -> {
           user.setStatus(UserStatus.ONLINE);
           userRepository.save(user);
       });

       return ResponseEntity.ok(new JwtResponse(
               jwt,
               userDetails.getId(),
               userDetails.getUsername(),
               userDetails.getEmail(),
               userDetails.getAvatarUrl()
       ));
   }
```

```
    @PostMapping("/register")
    public ResponseEntity<?> registerUser(@Valid @RequestBody SignupRequest signUpRequest) {
        if (userRepository.existsByUsername(signUpRequest.getUsername())) {
            Map<String, String> err = new HashMap<>();
            err.put("message", "Error: Username is already taken!");
            return ResponseEntity.badRequest().body(err);
        }

        if (userRepository.existsByEmail(signUpRequest.getEmail())) {
            Map<String, String> err = new HashMap<>();
            err.put("message", "Error: Email is already in use!");
            return ResponseEntity.badRequest().body(err);
        }

        User user = new User();
        user.setUsername(signUpRequest.getUsername());
        user.setEmail(signUpRequest.getEmail());
        user.setPassword(encoder.encode(signUpRequest.getPassword()));
        user.setStatus(UserStatus.ONLINE);
        user.setAvatarUrl(signUpRequest.getAvatarUrl());

        userRepository.save(user);

        Map<String, String> msg = new HashMap<>();
        msg.put("message", "User registered successfully!");
        return ResponseEntity.ok(msg);
    }

    @PostMapping("/logout")
    public ResponseEntity<?> logoutUser() {
        Authentication auth = SecurityContextHolder.getContext().getAuthentication();
        if (auth != null && auth.getPrincipal() instanceof UserDetailsImpl) {
            UserDetailsImpl userDetails = (UserDetailsImpl) auth.getPrincipal();
            userRepository.findByUsername(userDetails.getUsername()).ifPresent(user -> {
                user.setStatus(UserStatus.OFFLINE);
                userRepository.save(user);
            });
        }
        SecurityContextHolder.clearContext();
        Map<String, String> msg = new HashMap<>();
        msg.put("message", "Logged out successfully!");
        return ResponseEntity.ok(msg);
    }
}
```

LoginRequest.java

```
package com.chatapp.dto;
```

```java
import jakarta.validation.constraints.NotBlank;

public class LoginRequest {
    @NotBlank
    private String username;

    @NotBlank
    private String password;

    public LoginRequest() {}
    public LoginRequest(String username, String password) {
        this.username = username;
        this.password = password;
    }
    public String getUsername() { return username; }
    public void setUsername(String username) { this.username = username; }
    public String getPassword() { return password; }
    public void setPassword(String password) { this.password = password; }
}
```

SignupRequest.java

```java
package com.chatapp.dto;

import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;

public class SignupRequest {
    @NotBlank
    @Size(min = 3, max = 20)
    private String username;

    @NotBlank
    @Size(min = 6, max = 40)
    private String password;

    @NotBlank
    @Email
    private String email;

    private String avatarUrl;

    public SignupRequest() {}
    public SignupRequest(String username, String password, String email, String avatarUrl) {
        this.username = username;
        this.password = password;
        this.email = email;
        this.avatarUrl = avatarUrl;
    }
```

```
    public String getUsername() { return username; }
    public void setUsername(String username) { this.username = username; }
    public String getPassword() { return password; }
    public void setPassword(String password) { this.password = password; }
    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
    public String getAvatarUrl() { return avatarUrl; }
    public void setAvatarUrl(String avatarUrl) { this.avatarUrl = avatarUrl; }
}
```

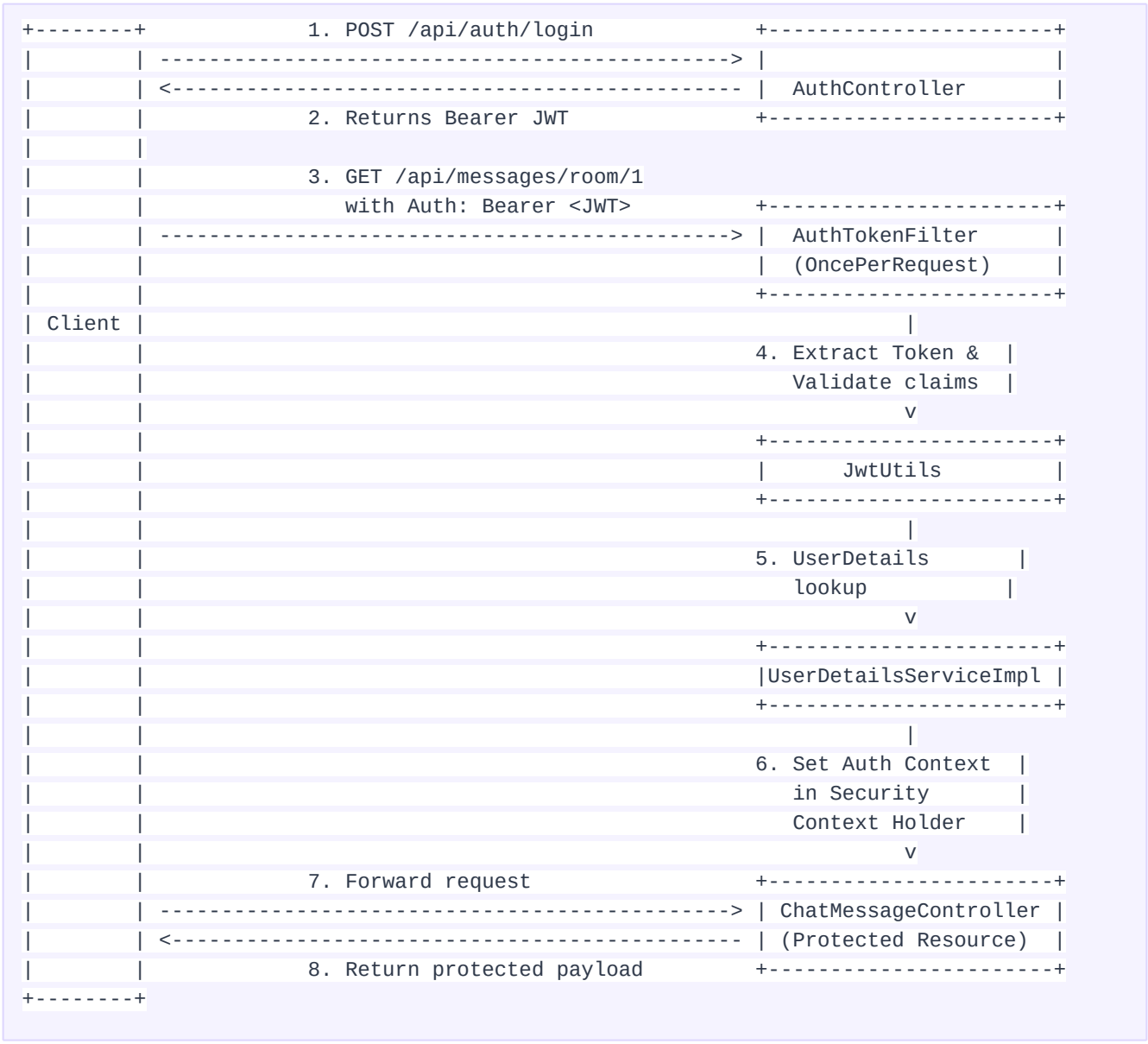
JwtResponse.java

```
package com.chatapp.dto;

public class JwtResponse {
    private String token;
    private String type = "Bearer";
    private Long id;
    private String username;
    private String email;
    private String avatarUrl;

    public JwtResponse() {}
    public JwtResponse(String token, Long id, String username, String email, String avatarUrl) {
        this.token = token;
        this.id = id;
        this.username = username;
        this.email = email;
        this.avatarUrl = avatarUrl;
    }
    public String getToken() { return token; }
    public void setToken(String token) { this.token = token; }
    public String getType() { return type; }
    public void setType(String type) { this.type = type; }
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getUsername() { return username; }
    public void setUsername(String username) { this.username = username; }
    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
    public String getAvatarUrl() { return avatarUrl; }
    public void setAvatarUrl(String avatarUrl) { this.avatarUrl = avatarUrl; }
}
```

---

6. JWT Authentication Lifecycle Diagram

Below is the execution pipeline showing how security checks intercept incoming HTTP requests:



---

7. Verification & Postman Testing Scenarios

Test Case 1: Fetching Identity Context Without Authorization Header

- **Request**: `GET http://localhost:8080/api/users/me` (No headers attached)
- **Expected Status**: `401 Unauthorized`
- **Response Payload**: `json { "status": 401, "error": "Unauthorized", "message": "Full authentication is required to access this resource", "path": "/api/users/me" }`

Test Case 2: Registering a New Account

- **Request**: `POST http://localhost:8080/api/auth/register`
- **Body Payload**: `json { "username": "demouser", "email": "demo@chatapp.com", "password": "demopassword123", "avatarUrl": "https://avatar.example.com/demo.png" }`
- **Expected Status**: `200 OK`
- **Response Payload**: `json { "message": "User registered successfully!" }`

Test Case 3: Requesting Bearer JWT Token

- **Request**: `POST http://localhost:8080/api/auth/login`
- **Body Payload**: `json { "username": "demouser", "password": "demopassword123" }`
- **Expected Status**: `200 OK`
- **Response Payload**: `json { "token": "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJkZW1vdXNlciIsImlhdCI6...", "type": "Bearer", "id": 1, "username": "demouser", "email": "demo@chatapp.com", "avatarUrl": "https://avatar.example.com/demo.png" }`

Test Case 4: Requesting Identity Context With Authorization Header

- **Request**: `GET http://localhost:8080/api/users/me`
- **Headers**: `Authorization: Bearer eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJkZW1vdXNlciIsImlhdCI6...`
- **Expected Status**: `200 OK`
- **Response Payload**: `json { "id": 1, "username": "demouser", "email": "demo@chatapp.com", "status": "ONLINE", "avatarUrl": "https://avatar.example.com/demo.png", "createdAt": "2026-06-07T22:45:00" }`

Test Case 5: Requesting Identity Context With Invalid Signature Signature

- **Request**: `GET http://localhost:8080/api/users/me`
- **Headers**: `Authorization: Bearer INVALID_TOKEN_SIGNATURE`
- **Expected Status**: `401 Unauthorized`
- **Response Payload**: `json { "status": 401, "error": "Unauthorized", "message": "Full authentication is required to access this resource", "path": "/api/users/me" }`